OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

   To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

```
  GNU nano 8.7.1                  skill1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process PID: %d\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        perror("exec");
        exit(1);
    } else {
        printf("Parent Process PID: %d\n", getpid());
        waitpid(pid, NULL, 0);
        printf("Child process completed\n");
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill1.c
hemaswi@Hemaswi:~$ gcc skill1.c -o skill1
hemaswi@Hemaswi:~$ ./skill1
Parent Process PID: 566
Child Process PID: 567
total 1004
-rwxr-xr-x 1 hemaswi hemaswi 70656 Sep 18 03:52 Demandings
-rw-r--r-- 1 hemaswi hemaswi   345 Sep 18 04:04 Demandings.c
drwxr-xr-x 2 hemaswi hemaswi  4096 Jul 28 04:06 Ex1
drwxr-xr-x 2 hemaswi hemaswi  4096 Aug  4 13:14 OSLab
drwxr-xr-x 6 hemaswi hemaswi  4096 Sep 29 03:04 OSSP_Project
drwxr-xr-x 2 hemaswi hemaswi  4096 Aug  4 13:10 OSlab
-rwxr-xr-x 1 hemaswi hemaswi 70568 Sep 18 03:51 Virtualaddress
-rw-r--r-- 1 hemaswi hemaswi   136 Sep 18 04:03 Virtualaddress.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 a
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:24 a.txt
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 b
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:16 b.txt
-rwxr-xr-x 1 hemaswi hemaswi 70584 Aug 14 09:46 childparentproce
ss
-rw-r--r-- 1 hemaswi hemaswi   228 Aug 14 09:45 childparentproce
ss.c
-rw-r--r-- 1 hemaswi hemaswi    22 Sep 29 14:17 data.txt
-rw-r--r-- 1 hemaswi hemaswi    56 Sep 29 14:09 destination.txt
-rwxr-xr-x 1 hemaswi hemaswi 70504 Jul 31 10:04 displayprocess
-rw-r--r-- 1 hemaswi hemaswi   110 Jul 31 10:03 displayprocess.c
-rw-r--r-- 1 hemaswi hemaswi     0 Jul 31 04:02 example.txt
-rwxr-xr-x 1 hemaswi hemaswi 70496 Aug 14 10:09 executelinux
-rw-r--r-- 1 hemaswi hemaswi   153 Aug 14 10:09 executelinux.c
-rw-r--r-- 1 hemaswi hemaswi    68 Jul 31 09:16 fil1.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:20 file.txt
-rwxr-xr-x 1 hemaswi hemaswi 70448 Aug  4 13:39 file1
-rw-r--r-- 1 hemaswi hemaswi    68 Jul 31 09:53 file1.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:13 file1.txt
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 file2
-rwxr-xr-x 1 hemaswi hemaswi 70488 Jul 31 10:06 fork
-rw-r--r-- 1 hemaswi hemaswi   104 Jul 31 10:06 fork.c
```

---

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

   To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
  GNU nano 8.7.1                  skill2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char command[100];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strlen(command) == 0)
            continue;

        printf("You entered: %s\n", command);
    }

    return 0;
}
hemaswi@Hemaswi:~$ nano skill2.c
hemaswi@Hemaswi:~$ gcc skill2.c -o skill2
hemaswi@Hemaswi:~$ ./skill2
myshell> hello
You entered: hello
myshell> ls
You entered: ls
myshell> exit
hemaswi@Hemaswi:~$
```

---

3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

   To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

GNU nano 8.7.1 skill3.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct Node {
    char command[MAX];
    struct Node *next;
} Node;
int main() {
    char **history = NULL;
    int count = 0, capacity = 2;
    char input[MAX];
    Node *head = NULL, *tail = NULL;
    history = malloc(capacity * sizeof(char *));
    if (history == NULL)
        return 1;
    while (1) {
        printf("myshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0)
            break;
        if (strlen(input) == 0)
            continue;
        if (count == capacity) {
            capacity *= 2;
            char **temp = realloc(history, capacity * sizeof(char *));
            if (temp == NULL)
                break;
            history = temp;
        }
        history[count] = malloc(strlen(input) + 1);
        if (history[count] == NULL)
            break;

        strcpy(history[count], input);
        count++;

        Node *newNode = malloc(sizeof(Node));
        if (newNode == NULL)
            break;

        strcpy(newNode->command, input);
        newNode->next = NULL;

        if (head == NULL)
            head = newNode;
        else
            tail->next = newNode;

        tail = newNode;

        printf("Command stored: %s\n", input);
    }

    printf("\nCommand History:\n");
    for (int i = 0; i < count; i++)
        printf("%d. %s\n", i + 1, history[i]);

    printf("\nLinked List History:\n");
    Node *current = head;

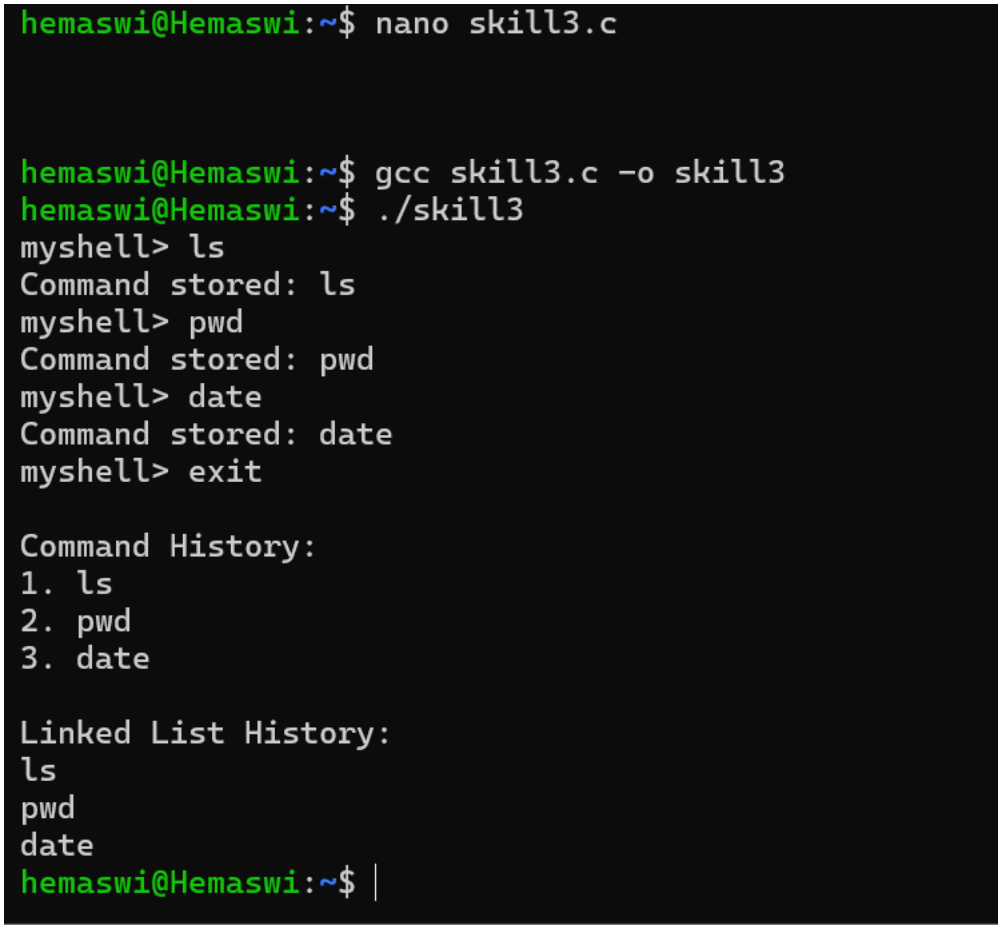
    while (current != NULL) {
        printf("%s\n", current->command);
        current = current->next;
    }

    for (int i = 0; i < count; i++)
        free(history[i]);

    free(history);

    while (head != NULL) {
        Node *temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```



---

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

   To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.



---

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

   To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
 GNU nano 8.7.1                                skill5.c *
#include <stdio.h>
#include <string.h>

int main() {
    char command[200];
    char token[200];
    int i = 0, j = 0;
    char quote = '\0';

    printf("myshell> ");

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 0;

    command[strcspn(command, "\n")] = '\0';

    while (command[i] != '\0') {
        if (command[i] == '\\' && command[i + 1] != '\0') {
            token[j++] = command[i + 1];
            i += 2;
        } else if (quote != '\0') {
            if (command[i] == quote)
                quote = '\0';
            else
                token[j++] = command[i];
            i++;
        } else if (command[i] == '\'' || command[i] == '"') {
            quote = command[i++];
        } else if (command[i] == ' ' || command[i] == '\t') {
            if (j > 0) {
                token[j] = '\0';
                printf("%s\n", token);
                j = 0;
            }
            i++;
        } else {
            token[j++] = command[i++];
        }
    }

    if (j > 0) {
        token[j] = '\0';
        printf("%s\n", token);
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill5.c
hemaswi@Hemaswi:~$ gcc skill5.c -o skill5
hemaswi@Hemaswi:~$ ./skill5
myshell> echo "hello world" 'OSSP lab'
echo
hello world
OSSP lab
hemaswi@Hemaswi:~$
```

---

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

   To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

GNU nano 8.7.1 skill6.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[200];
    char *args[50];
    int i = 0;

    printf("myshell> ");

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 0;

    command[strcspn(command, "\n")] = '\0';

    char *token = strtok(command, " \t");

    while (token != NULL && i < 49) {
        args[i++] = token;
        token = strtok(NULL, " \t");
    }

    args[i] = NULL;

    if (i == 0)
        return 0;

    if (strcmp(args[0], "exit") == 0)
        return 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        execvp(args[0], args);
        perror("execvp");
        _exit(127);
    } else {
        int status;
        waitpid(pid, &status, 0);
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill6.c
hemaswi@Hemaswi:~$ gcc skill6.c -o skill6
hemaswi@Hemaswi:~$ ./skill6
myshell> ls -l
total 1108
-rwxr-xr-x 1 hemaswi hemaswi 70656 Sep 18 03:52 Demandings
-rw-r--r-- 1 hemaswi hemaswi   345 Sep 18 04:04 Demandings.c
drwxr-xr-x 2 hemaswi hemaswi  4096 Jul 28 04:06 Ex1
drwxr-xr-x 2 hemaswi hemaswi  4096 Aug  4 13:14 OSLab
drwxr-xr-x 6 hemaswi hemaswi  4096 Sep 29 03:04 OSSP_Project
drwxr-xr-x 2 hemaswi hemaswi  4096 Aug  4 13:10 OSlab
-rwxr-xr-x 1 hemaswi hemaswi 70568 Sep 18 03:51 Virtualaddress
-rw-r--r-- 1 hemaswi hemaswi   136 Sep 18 04:03 Virtualaddress.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 a
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:24 a.txt
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 b
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:16 b.txt
-rwxr-xr-x 1 hemaswi hemaswi 70584 Aug 14 09:46 childparentprocess
-rw-r--r-- 1 hemaswi hemaswi   228 Aug 14 09:45 childparentprocess.c
-rw-r--r-- 1 hemaswi hemaswi    22 Sep 29 14:17 data.txt
-rw-r--r-- 1 hemaswi hemaswi    56 Sep 29 14:09 destination.txt
-rwxr-xr-x 1 hemaswi hemaswi 70504 Jul 31 10:04 displayprocess
-rw-r--r-- 1 hemaswi hemaswi   110 Jul 31 10:03 displayprocess.c
-rw-r--r-- 1 hemaswi hemaswi     0 Jul 31 04:02 example.txt
-rwxr-xr-x 1 hemaswi hemaswi 70496 Aug 14 10:09 executelinux
-rw-r--r-- 1 hemaswi hemaswi   153 Aug 14 10:09 executelinux.c
-rw-r--r-- 1 hemaswi hemaswi    68 Jul 31 09:16 fil1.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:20 file.txt
-rwxr-xr-x 1 hemaswi hemaswi 70448 Aug  4 13:39 file1
-rw-r--r-- 1 hemaswi hemaswi    68 Jul 31 09:53 file1.c
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:13 file1.txt
-rw-r--r-- 1 hemaswi hemaswi     0 Aug  4 13:39 file2
-rwxr-xr-x 1 hemaswi hemaswi 70488 Jul 31 10:06 fork
-rw-r--r-- 1 hemaswi hemaswi   104 Jul 31 10:06 fork.c
-rwxr-xr-x 1 hemaswi hemaswi 70576 Aug  4 12:58 forkexec
-rw-r--r-- 1 hemaswi hemaswi   264 Aug  4 12:57 forkexec.c
-rwxr-xr-x 1 hemaswi hemaswi 70448 Aug  4 13:28 hello
-rw-r--r-- 1 hemaswi hemaswi    67 Aug  4 13:26 hello.c
-rw-r--r-- 1 hemaswi hemaswi    31 Sep 29 14:13 input.txt
-rw-r--r-- 1 hemaswi hemaswi     0 Sep 22 04:51 main.c
-rwxr-xr-x 1 hemaswi hemaswi 70616 Sep 18 03:57 malloc
-rw-r--r-- 1 hemaswi hemaswi   382 Sep 18 04:07 malloc.c
-rwxr-xr-x 1 hemaswi hemaswi 70464 Sep 18 03:59 memory_accessing
-rw-r--r-- 1 hemaswi hemaswi   142 Sep 18 04:08 memory_accessing.c
-rwxr-xr-x 1 hemaswi hemaswi 70552 Aug 14 10:04 multiplechildprocesses
-rw-r--r-- 1 hemaswi hemaswi   160 Aug 14 10:04 multiplechildprocesses.c
-rw-r--r-- 1 hemaswi hemaswi    31 Sep 29 14:13 output.txt
-rwxr-xr-x 1 hemaswi hemaswi 70496 Aug  4 12:42 parentchild
-rw-r--r-- 1 hemaswi hemaswi   230 Aug  4 12:40 parentchild.c
-rwxr-xr-x 1 hemaswi hemaswi 70600 Aug 14 10:15 parentchildexecutingconcurrently
-rw-r--r-- 1 hemaswi hemaswi   152 Aug 14 10:15 parentchildexecutingconcurrently.c
-rwxr-xr-x 1 hemaswi hemaswi 70584 Aug  4 12:50 parentchildpid
-rw-r--r-- 1 hemaswi hemaswi   287 Aug  4 12:52 parentchildpid.c
-rwxr-xr-x 1 hemaswi hemaswi 70584 Aug 14 09:57 parentwaitchild
-rw-r--r-- 1 hemaswi hemaswi   260 Aug 14 09:57 parentwaitchild.c
-rwxr-xr-x 1 hemaswi hemaswi 70568 Sep 18 03:53 physical_address
-rw-r--r-- 1 hemaswi hemaswi   584 Sep 18 04:11 physical_address.c
-rwxr-xr-x 1 hemaswi hemaswi 70784 Sep 29 03:39 praactical3
-rw-r--r-- 1 hemaswi hemaswi   999 Sep 29 03:39 praactical3.c
-rwxr-xr-x 1 hemaswi hemaswi 70856 Sep 29 03:33 practical1
-rw-r--r-- 1 hemaswi hemaswi   558 Sep 29 03:33 practical1.c
-rwxr-xr-x 1 hemaswi hemaswi 70928 Sep 29 14:17 practical10
-rw-r--r-- 1 hemaswi hemaswi  1017 Sep 29 14:17 practical10.c
-rwxr-xr-x 1 hemaswi hemaswi 70752 Sep 29 14:19 practical11A
-rw-r--r-- 1 hemaswi hemaswi   625 Sep 29 14:18 practical11A.c
```

---

7. Process Synchronization and Child Process Monitoring Using waitpid().

   To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```
 GNU nano 8.7.1                                  skill7.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[200];
    char *args[50];
    int i = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 0;

    command[strcspn(command, "\n")] = '\0';

    char *token = strtok(command, " \t");

    while (token != NULL && i < 49) {
        args[i++] = token;
        token = strtok(NULL, " \t");
    }
    args[i] = NULL;

    if (i == 0)
        return 0;

    if (strcmp(args[0], "exit") == 0)
        return 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        execvp(args[0], args);
        perror("Command execution failed");
        _exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status))
        printf("Exit status: %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Terminated by signal: %d\n", WTERMSIG(status));

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill7.c
hemaswi@Hemaswi:~$ gcc skill7.c -o skill7
hemaswi@Hemaswi:~$ ./skill7
myshell> pwd
/home/hemaswi
Exit status: 0
hemaswi@Hemaswi:~$
```

---

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

   To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
 GNU nano 8.7.1                                                skill8.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char command[200];
    char *args[50];
    int i;

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        i = 0;
        char *token = strtok(command, " \t");

        while (token != NULL && i < 49) {
            args[i++] = token;
            token = strtok(NULL, " \t");
        }
        args[i] = NULL;

        if (i == 0)
            continue;

        if (strcmp(args[0], "exit") == 0)
            break;

        if (strcmp(args[0], "cd") == 0) {
            char *path = args[1] ? args[1] : getenv("HOME");

            if (path == NULL || chdir(path) != 0)
                perror("cd");
            continue;
        }

        if (strcmp(args[0], "pwd") == 0) {
            char cwd[1024];

            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("%s\n", cwd);
            else
                perror("pwd");

            continue;
        }

        if (strcmp(args[0], "echo") == 0) {
            for (int j = 1; args[j] != NULL; j++) {
                char *p = args[j];

                if (p[0] == '$') {
                    char *value = getenv(p + 1);
                    printf("%s", value ? value : "");
                } else {
                    printf("%s", p);
                }

                if (args[j + 1] != NULL)
                    printf(" ");
            }

            printf("\n");
            continue;
        }

        printf("Unknown built-in command: %s\n", args[0]);
    }

    return 0;
```

```
hemaswi@Hemaswi:~$ nano skill8.c
hemaswi@Hemaswi:~$ gcc skill8.c -o skill8
hemaswi@Hemaswi:~$ ./skill8
\myshell> pwd
Unknown built-in command: \pwd
myshell> echo $HOME
/home/hemaswi
myshell> cd /
myshell> pwd
/
myshell> exit
```

---

9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

   To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.



```
 GNU nano 8.7.1                                  skill9.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char command[200];
    char *args[20];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        int i = 0;
        char *token = strtok(command, " \t");

        while (token != NULL && i < 19) {
            args[i++] = token;
            token = strtok(NULL, " \t");
        }
        args[i] = NULL;

        if (i == 0)
            continue;

        if (strcmp(args[0], "exit") == 0)
            break;

        if (strcmp(args[0], "cd") == 0) {
            char *path = args[1] ? args[1] : getenv("HOME");

            if (path == NULL || chdir(path) != 0) {
                perror("cd");
            }
            continue;
        }

        if (strcmp(args[0], "pwd") == 0) {
            char cwd[1024];

            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("%s\n", cwd);
            else
                perror("pwd");

            continue;
        }

        printf("Unsupported command: %s\n", args[0]);
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill9.c
hemaswi@Hemaswi:~$ gcc skill9.c -o skill9
hemaswi@Hemaswi:~$ ./skill9
myshell> pwd
/home/hemaswi
myshell> cd /
myshell> pwd
/
myshell> exit
hemaswi@Hemaswi:~$
```

---

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

    To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
  GNU nano 8.7.1                                  skill10.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[100];
    char value[200];
    char *env;

    printf("Enter environment variable name: ");
    if (fgets(name, sizeof(name), stdin) == NULL)
        return 1;

    name[strcspn(name, "\n")] = '\0';

    env = getenv(name);

    if (env != NULL)
        printf("Value: %s\n", env);
    else
        printf("Environment variable not found.\n");

    printf("Enter new value: ");
    if (fgets(value, sizeof(value), stdin) == NULL)
        return 1;

    value[strcspn(value, "\n")] = '\0';

    if (setenv(name, value, 1) == 0)
        printf("Updated value: %s\n", getenv(name));
    else
        perror("setenv");

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill10.c
hemaswi@Hemaswi:~$ gcc skill10.c -o skill10
hemaswi@Hemaswi:~$ ./skill10
Enter environment variable name: HOME
Value: /home/hemaswi
Enter new value: /tmp
Updated value: /tmp
hemaswi@Hemaswi:~$
```

---

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

    To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.



```
 GNU nano 8.7.1                                  skill11.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 100
#define MAX_LENGTH 200

int main() {
    char history[MAX_COMMANDS][MAX_LENGTH];
    char command[MAX_LENGTH];
    int count = 0;

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            for (int i = 0; i < count; i++)
                printf("%d  %s\n", i + 1, history[i]);
            continue;
        }

        if (strlen(command) == 0)
            continue;

        if (count < MAX_COMMANDS) {
            strcpy(history[count], command);
            count++;
        } else {
            memmove(history, history + 1,
                    (MAX_COMMANDS - 1) * sizeof(history[0]));
            strcpy(history[MAX_COMMANDS - 1], command);
        }
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill11.c
hemaswi@Hemaswi:~$ gcc skill11.c -o skill11
hemaswi@Hemaswi:~$ ./skill11
myshell> ls
myshell> pwd
myshell> history
1  ls
2  pwd
myshell> exit
hemaswi@Hemaswi:~$
```

---

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

    To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
 GNU nano 8.7.1                                                  ski
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char input[200];
    char *cmd1[50], *cmd2[50];
    int fd[2], i = 0, j = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    char *pipe_pos = strchr(input, '|');

    if (pipe_pos == NULL) {
        fprintf(stderr, "Use | between two commands.\n");
        return 1;
    }

    *pipe_pos = '\0';

    char *token = strtok(input, " \t");
    while (token != NULL && i < 49) {
        cmd1[i++] = token;
        token = strtok(NULL, " \t");
    }
    cmd1[i] = NULL;

    token = strtok(pipe_pos + 1, " \t");
    while (token != NULL && j < 49) {
        cmd2[j++] = token;
        token = strtok(NULL, " \t");
    }
    cmd2[j] = NULL;

    if (i == 0 || j == 0) {
        fprintf(stderr, "Invalid command.\n");
        return 1;
    }

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return 1;
    }

    if (p1 == 0) {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execvp(cmd1[0], cmd1);
        perror("execvp");
        _exit(127);
    }

    pid_t p2 = fork();

    if (p2 == -1) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        waitpid(p1, NULL, 0);
        return 1;
    }

    if (p2 == 0) {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execvp(cmd2[0], cmd2);
        perror("execvp");
        _exit(127);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

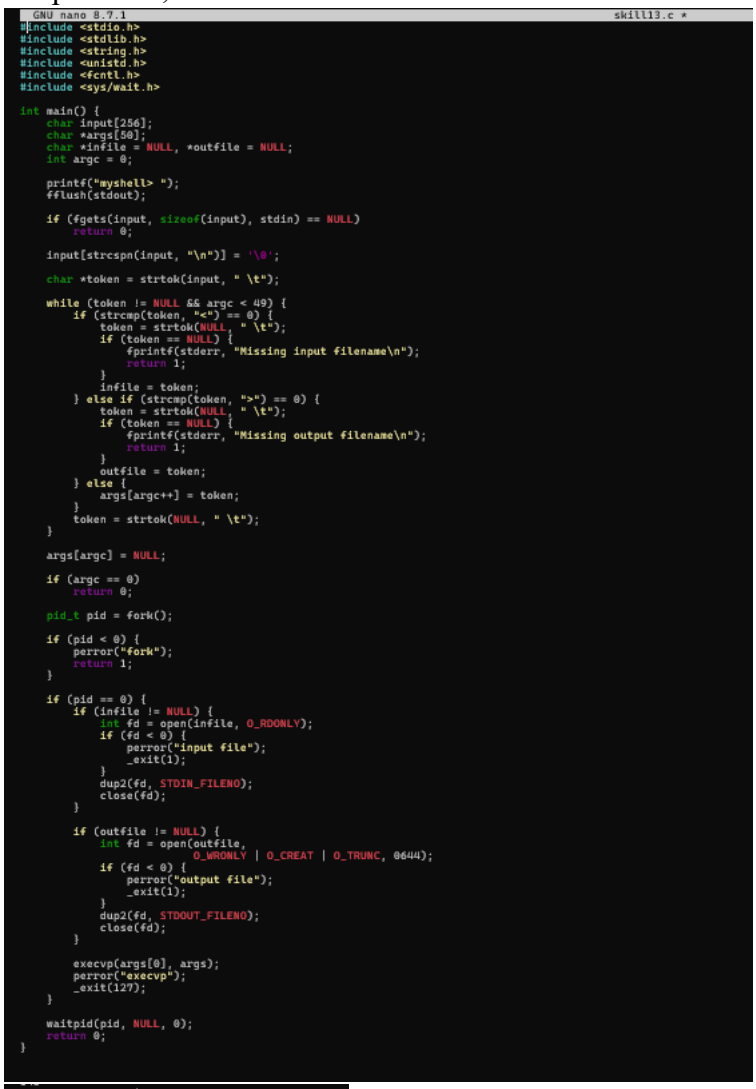
    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill12.c
```

```
hemaswi@Hemaswi:~$ gcc skill12.c -o skill12
hemaswi@Hemaswi:~$ ./skill12
myshell> ls | wc -l
143
hemaswi@Hemaswi:~$
```

13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

    To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.



```
 GNU nano 8.7.1                                                skill13.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    char input[256];
    char *args[50];
    char *infile = NULL, *outfile = NULL;
    int argc = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && argc < 49) {
        if (strcmp(token, "<") == 0) {
            token = strtok(NULL, " \t");
            if (token == NULL) {
                fprintf(stderr, "Missing input filename\n");
                return 1;
            }
            infile = token;
        } else if (strcmp(token, ">") == 0) {
            token = strtok(NULL, " \t");
            if (token == NULL) {
                fprintf(stderr, "Missing output filename\n");
                return 1;
            }
            outfile = token;
        } else {
            args[argc++] = token;
        }
        token = strtok(NULL, " \t");
    }

    args[argc] = NULL;

    if (argc == 0)
        return 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        if (infile != NULL) {
            int fd = open(infile, O_RDONLY);
            if (fd < 0) {
                perror("input file");
                _exit(1);
            }
            dup2(fd, STDIN_FILENO);
            close(fd);
        }

        if (outfile != NULL) {
            int fd = open(outfile,
                          O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd < 0) {
                perror("output file");
                _exit(1);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        execvp(args[0], args);
        perror("execvp");
        _exit(127);
    }

    waitpid(pid, NULL, 0);
    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill13.c



hemaswi@Hemaswi:~$ gcc skill13.c -o skill13
hemaswi@Hemaswi:~$ ./skill13
myshell> ls > files.txt
hemaswi@Hemaswi:~$
```

---

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

    To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```
 GNU nano 8.7.1                                                skill14.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    char input[200];
    char *args[50];
    char *outfile = NULL;
    int i = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && i < 49) {
        if (strcmp(token, ">") == 0) {
            token = strtok(NULL, " \t");

            if (token == NULL) {
                fprintf(stderr, "Missing output filename\n");
                return 1;
            }

            outfile = token;
        } else {
            args[i++] = token;
        }

        token = strtok(NULL, " \t");
    }

    args[i] = NULL;

    if (i == 0 || outfile == NULL) {
        fprintf(stderr, "Usage: command > filename\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        int fd = open(outfile,
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (fd < 0) {
            perror("open");
            _exit(1);
        }

        if (dup2(fd, STDOUT_FILENO) < 0) {
            perror("dup2");
            close(fd);
            _exit(1);
        }

        close(fd);

        execvp(args[0], args);
        perror("execvp");
        _exit(127);
    }

    waitpid(pid, NULL, 0);
    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill14.c

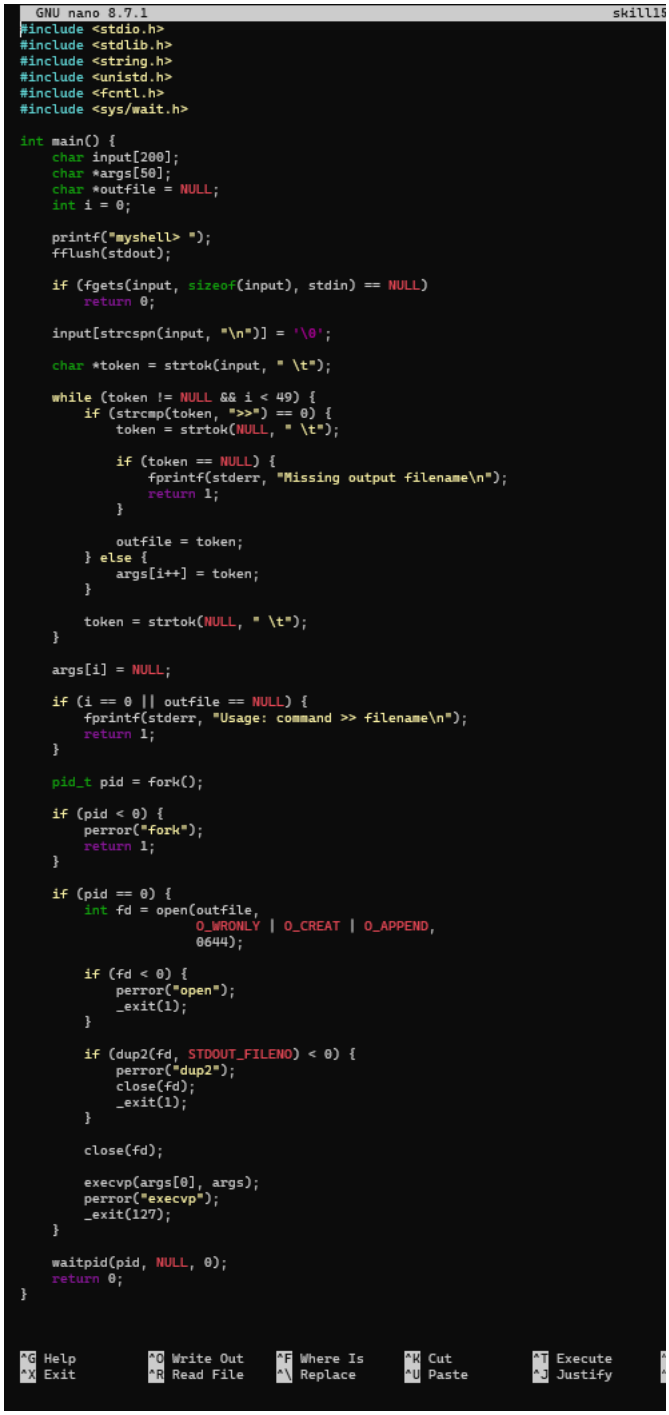


hemaswi@Hemaswi:~$ gcc skill14.c -o skill14
./skill14
myshell> ls
Usage: command > filename
hemaswi@Hemaswi:~$ gcc skill14.c -o skill14
./skill14
myshell> ls > output.txt
hemaswi@Hemaswi:~$
```

---

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

    To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.


GNU nano 8.7.1 skill15
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main() {
char input[200];
char *args[50];
char *outfile = NULL;
int i = 0;
printf("myshell> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
return 0;
input[strcspn(input, "\n")] = '\0';
char *token = strtok(input, " \t");
while (token != NULL && i < 49) {
if (strcmp(token, ">>") == 0) {
token = strtok(NULL, " \t");
if (token == NULL) {
fprintf(stderr, "Missing output filename\n");
return 1;
}
outfile = token;
} else {
args[i++] = token;
}
token = strtok(NULL, " \t");
}
args[i] = NULL;
if (i == 0 || outfile == NULL) {
fprintf(stderr, "Usage: command >> filename\n");
return 1;
}
pid_t pid = fork();
if (pid < 0) {
perror("fork");
return 1;
}
if (pid == 0) {
int fd = open(outfile,
O_WRONLY | O_CREAT | O_APPEND,
0644);
if (fd < 0) {
perror("open");
_exit(1);
}
if (dup2(fd, STDOUT_FILENO) < 0) {
perror("dup2");
close(fd);
_exit(1);
}
close(fd);
execvp(args[0], args);
perror("execvp");
_exit(127);
}
waitpid(pid, NULL, 0);
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify



---

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

    To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.



```
 GNU nano 8.7.1                                  skill16.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    char input[200];
    char *args[50];
    char *infile = NULL;
    int i = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && i < 49) {
        if (strcmp(token, "<") == 0) {
            token = strtok(NULL, " \t");

            if (token == NULL) {
                fprintf(stderr, "Missing input filename\n");
                return 1;
            }

            infile = token;
        } else {
            args[i++] = token;
        }

        token = strtok(NULL, " \t");
    }

    args[i] = NULL;

    if (i == 0 || infile == NULL) {
        fprintf(stderr, "Usage: command < filename\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        int fd = open(infile, O_RDONLY);

        if (fd < 0) {
            perror("open");
            _exit(1);
        }

        if (dup2(fd, STDIN_FILENO) < 0) {
            perror("dup2");
            close(fd);
            _exit(1);
        }

        close(fd);

        execvp(args[0], args);
        perror("execvp");
        _exit(127);
    }

    waitpid(pid, NULL, 0);
    return 0;
}

^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

```
hemaswi@Hemaswi:~$ nano skill16.c
hemaswi@Hemaswi:~$ gcc skill16.c -o skill16
./skill16
myshell> cat < input.txt
Hello, this is my input file.
```

---

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

    To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.



```
 GNU nano 8.7.1                                          skill17.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    char input[200];
    char *args[50];
    char *errfile = NULL;
    int i = 0;

    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && i < 49) {
        if (strcmp(token, "2>") == 0) {
            token = strtok(NULL, " \t");

            if (token == NULL) {
                fprintf(stderr, "Missing error filename\n");
                return 1;
            }

            errfile = token;
        } else {
            args[i++] = token;
        }

        token = strtok(NULL, " \t");
    }

    args[i] = NULL;

    if (i == 0 || errfile == NULL) {
        fprintf(stderr, "Usage: command 2> filename\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        int fd = open(errfile,
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (fd < 0) {
            perror("open");
            _exit(1);
        }

        if (dup2(fd, STDERR_FILENO) < 0) {
            perror("dup2");
            close(fd);
            _exit(1);
        }

        close(fd);

        execvp(args[0], args);
        perror("execvp");
        _exit(127);
    }

    waitpid(pid, NULL, 0);
    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill17.c
hemaswi@Hemaswi:~$ gcc skill17.c -o skill1
./skill17
myshell> ls nonexistent_file 2> errors.txt
```

---

18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

    To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
 GNU nano 8.7.1                                skill18.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char input[200];
    char *args[50];
    int background, i;

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        i = 0;
        char *token = strtok(input, " \t");

        while (token != NULL && i < 49) {
            args[i++] = token;
            token = strtok(NULL, " \t");
        }
        args[i] = NULL;

        if (i == 0)
            continue;

        if (strcmp(args[0], "exit") == 0)
            break;

        background = 0;

        if (strcmp(args[i - 1], "&") == 0) {
            background = 1;
            args[--i] = NULL;
        }

        if (i == 0)
            continue;

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {
            execvp(args[0], args);
            perror("execvp");
            _exit(127);
        }

        if (background) {
            printf("Started background process: %d\n", pid);
        } else {
            int status;
            if (waitpid(pid, &status, 0) == -1)
                perror("waitpid");
        }
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill18.c
hemaswi@Hemaswi:~$ gcc skill18.c -o skill18
hemaswi@Hemaswi:~$ ./skill18
myshell> sleep 10 &
Started background process: 1604
myshell> date
Tue Oct 13 03:36:59 UTC 2026
myshell> exit
hemaswi@Hemaswi:~$
```

---

19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

    To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
 GNU nano 8.7.1                                skill19.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t running = 1;

void handle_signal(int sig) {
    if (sig == SIGINT) {
        running = 0;
    }
}

int main() {
    struct sigaction sa;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C to stop the program.\n");

    while (running) {
        printf("Running...\n");
        sleep(1);
    }

    printf("\nSIGINT received. Exiting safely.\n");
    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill19.c
hemaswi@Hemaswi:~$ gcc skill19.c -o skill19
./skill19
Process ID: 1562
Press Ctrl+C to stop the program.
Running...
Running...
Running...
Running...
Running...
^C
SIGINT received. Exiting safely.
hemaswi@Hemaswi:~$
```

---

20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

    To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
 GNU nano 8.7.1                                  skill20.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

int main() {
    printf("Process ID: %d\n", getpid());
    printf("Process started. Press Ctrl+Z to suspend it.\n");
    printf("Use the shell command 'fg' to resume it.\n");

    while (1) {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill20.c
hemaswi@Hemaswi:~$ gcc skill20.c -o skill20
hemaswi@Hemaswi:~$ ./skill20
Process ID: 1650
Process started. Press Ctrl+Z to suspend it
Use the shell command 'fg' to resume it.
Process is running...
Process is running...
Process is running...
^Z
[1]+  Stopped                 ./skill20
hemaswi@Hemaswi:~$
```

---

21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

    To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
 GNU nano 8.7.1                                  skill21.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", getpgrp());

        while (1) {
            printf("Child process running...\n");
            sleep(2);
        }
    } else {
        sleep(1);

        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Sending SIGTERM to child process group...\n");
        if (kill(-pid, SIGTERM) == -1)
            perror("kill");

        waitpid(pid, NULL, 0);
        printf("Child process terminated.\n");
    }

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill21.c
hemaswi@Hemaswi:~$ gcc skill21.c -o skill21
./skill21
Child PID: 1688
Process Group ID: 1688
Child process running...
Parent PID: 1687
Child PID: 1688
Sending SIGTERM to child process group...
Child process terminated.
hemaswi@Hemaswi:~$
```

---

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

    To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

GNU nano 8.7.1 skill22.c *

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, new_n;
    int *arr;

    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid size.\n");
        return 1;
    }

    arr = malloc(n * sizeof(int));

    if (arr == NULL) {
        perror("malloc");
        return 1;
    }

    printf("Enter %d elements:\n", n);

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            printf("Invalid input.\n");
            free(arr);
            return 1;
        }
    }

    printf("Original array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\nEnter new number of elements: ");
    if (scanf("%d", &new_n) != 1 || new_n <= 0) {
        printf("Invalid size.\n");
        free(arr);
        return 1;
    }

    int *temp = realloc(arr, new_n * sizeof(int));

    if (temp == NULL) {
        perror("realloc");
        free(arr);
        return 1;
    }

    arr = temp;

    if (new_n > n) {
        printf("Enter %d additional elements:\n", new_n - n);

        for (int i = n; i < new_n; i++) {
            if (scanf("%d", &arr[i]) != 1) {
                printf("Invalid input.\n");
                free(arr);
                return 1;
            }
        }
    }

    printf("Resized array:\n");
    for (int i = 0; i < new_n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);
    printf("Memory released successfully.\n");

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill22.c
hemaswi@Hemaswi:~$ gcc skill22.c -o skill22
./skill22
Enter number of elements: 3
Enter 3 elements:
10 20 30
Original array:
10 20 30
Enter new number of elements: 5
Enter 2 additional elements:
40 50
Resized array:
10 20 30 40 50
Memory released successfully.
hemaswi@Hemaswi:~$
```

---

23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

    To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
  GNU nano 8.7.1                                skill23.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int n;
    int *arr;

    printf("Enter number of elements: ");

    if (scanf("%d", &n) != 1 || n <= 0) {
        fprintf(stderr, "Error: Invalid number of elements.\n");
        return 1;
    }

    arr = malloc((size_t)n * sizeof(int));

    if (arr == NULL) {
        perror("Memory allocation failed");
        return 1;
    }

    printf("Enter %d integers:\n", n);

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            fprintf(stderr, "Error: Invalid integer input.\n");
            free(arr);
            return 1;
        }
    }

    printf("Entered elements:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

```
hemaswi@Hemaswi:~$ nano skill23.c
hemaswi@Hemaswi:~$ gcc skill23.c -o skill23
hemaswi@Hemaswi:~$ ./skill23
Enter number of elements: 5
Enter 5 integers:
1 5 6 9 4
Entered elements:
1 5 6 9 4
Memory released successfully.
hemaswi@Hemaswi:~$ valgrind --leak-check=full ./skill23
==2877== Memcheck, a memory error detector
==2877== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2877== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2877== Command: ./skill23
==2877==
Enter number of elements: 5
Enter 5 integers:
1 5 6 9 4
Entered elements:
1 5 6 9 4
Memory released successfully.
==2877==
==2877== HEAP SUMMARY:
==2877==     in use at exit: 0 bytes in 0 blocks
==2877==   total heap usage: 3 allocs, 3 frees, 2,068 bytes allocated
==2877==
==2877== All heap blocks were freed -- no leaks are possible
==2877==
==2877== For lists of detected and suppressed errors, rerun with: -s
==2877== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
hemaswi@Hemaswi:~$
```

---

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

    To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
 GNU nano 8.7.1                                  skill24.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    char input[256];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "pwd") == 0) {
            char cwd[1024];
            if (getcwd(cwd, sizeof(cwd)))
                printf("%s\n", cwd);
            else
                perror("pwd");
            continue;
        }

        if (strncmp(input, "cd ", 3) == 0) {
            if (chdir(input + 3) == -1)
                perror("cd");
            continue;
        }

        char *left = input;
        char *right = strchr(input, '|');
        int pipefd[2];
        pid_t pid1, pid2;

        if (right != NULL) {
            *right = '\0';
            right++;

            char *cmd1[50], *cmd2[50];
            int i = 0, j = 0;

            char *token = strtok(left, " \t");
            while (token && i < 49) {
                cmd1[i++] = token;
                token = strtok(NULL, " \t");
            }
            cmd1[i] = NULL;

            token = strtok(right, " \t");
            while (token && j < 49) {
                cmd2[j++] = token;
                token = strtok(NULL, " \t");
            }
```


GNU nano 8.7.1 skill24.c *
cmd2[j++] = token;
token = strtok(NULL, " \t");
}
cmd2[j] = NULL;
if (i == 0 || j == 0) {
fprintf(stderr, "Invalid pipe command\n");
continue;
}
if (pipe(pipefd) == -1) {
perror("pipe");
continue;
}
pid1 = fork();
if (pid1 == 0) {
dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[0]);
close(pipefd[1]);
execvp(cmd1[0], cmd1);
perror("execvp");
_exit(127);
}
pid2 = fork();
if (pid2 == 0) {
dup2(pipefd[0], STDIN_FILENO);
close(pipefd[0]);
close(pipefd[1]);
execvp(cmd2[0], cmd2);
perror("execvp");
_exit(127);
}
close(pipefd[0]);
close(pipefd[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
continue;
}
char *args[50];
int i = 0;
char *token = strtok(input, " \t");
while (token && i < 49) {
args[i++] = token;
token = strtok(NULL, " \t");
}
args[i] = NULL;
if (i == 0)
continue;
pid_t pid = fork();
if (pid == 0) {
execvp(args[0], args);
perror("execvp");
_exit(127);
} else if (pid > 0) {
waitpid(pid, NULL, 0);
} else {
perror("fork");
}
}
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

```
hemaswi@Hemaswi:~$ nano skill24.c
hemaswi@Hemaswi:~$ gcc -Wall -Wextra skill24.c -o skill24
./skill24
myshell> pwd
/home/hemaswi
myshell> ls
Demandings                          practical6B.c
Demandings.c                        practical6C
Ex1                                 practical6C.c
OSLab                               practical7
OSSP_Project                        practical7.c
OSlab                               practical8
Virtualaddress                      practical8.c
Virtualaddress.c                    practical8B
a                                   practical8B.c
a.txt                               practical9A
b                                   practical9A.c
b.txt                               practical9B
childparentprocess                  practical9B.c
childparentprocess.c                practical9C
data.txt                            practical9C.c
destination.txt                     practicall2
displayprocess                      practicall2.c
displayprocess.c                    priority_scheduler
errors.txt                          priority_scheduler_backup
example.txt                         process_making
executelinux                        process_making.c
executelinux.c                      processid
fil1.c                              processid.c
file.txt                            readuser
file1                               readuser.c
file1.c                             s2commands
file1.txt                           script
file2                               script.sh
files.txt                           sh
fork                                shortcut
fork.c                              skill1
forkexec                            skill1.c
forkexec.c                          skill10
hello                               skill10.c
hello.c                             skill11
input.txt                           skill11.c
main.c                              skill12
malloc                              skill12.c
malloc.c                            skill13
memory_accessing                    skill13.c
memory_accessing.c                  skill14
multiplechildprocesses              skill14.c
multiplechildprocesses.c            skill15
output.txt                          skill15.c
parentchild                         skill16
parentchild.c                       skill16.c
parentchildexecutingconcurrently    skill17
parentchildexecutingconcurrently.c  skill17.c
parentchildpid                      skill18
parentchildpid.c                    skill18.c
parentwaitchild                     skill19
parentwaitchild.c                   skill19.c
physical_address                    skill2
physical_address.c                  skill2.c
praactical3                         skill20
praactical3.c                       skill20.c
practical1                          skill21
practical1.c                        skill21.c
practical10                         skill22
practical10.c                       skill22.c
practical11A                        skill23
practical11A.c                      skill23.c
practical11B                        skill24
practical11B.c                      skill24.c
practical12                         skill3
practical12A                        skill3.c
practical12A.c                      skill4
practical12B                        skill4.c
practical12B.c                      skill5
practical12C.c                      skill5.c
practical2                          skill6
practical2.c                        skill6.c
practical4                          skill7
practical4.c                        skill7.c
practical5A                         skill8
practical5A.c                       skill8.c
practical5B                         skill9
practical5B.c                       skill9.c
practical5C                         source.txt
practical5C.c                       virtualadress.c.save
practical6                          wait
practical6.c                        wait.c
practical6A                         waiting
practical6A.c                       waiting.c
practical6B
myshell> ls | wc -l
169
myshell> cd /
myshell> pwd
/
myshell> exit
hemaswi@Hemaswi:~$ ~
```